

CRYPTO HAWKING · ETHDENVER 2025 WINNER

Smart Contract Security Audit

Morpho

0xBBBbBBb9cC5e90e3b3Af64bdAF62C37EEFFCb

Ethereum

Compiler: v0.8.19+commit.7dd6d404

Date: October 7, 2026

OVERALL RISK RATING *AI-assisted audit. Not a substitute for manual review.*

Low

Executive Summary

Independent security review of publicly verified source, not commissioned by the project. This is the canonical Morpho Blue core lending contract, a minimal, immutable, and extensively audited singleton. Core accounting (supply/withdraw/borrow/repay/liquidate/flashLoan) correctly follows checks-effects-interactions, uses virtual-shares accounting to mitigate inflation attacks, and keeps the owner role intentionally limited (no mint/pause/drain capability). Findings are limited to documented design trust-assumptions (unvalidated owner/fee-recipient setters, oracle/IRM trust, fixed EIP-712 domain separator) and minor style issues rather than exploitable logic flaws.

Findings (6)

F-001

Low

setOwner allows setting owner to the zero address, permanently bricking all onlyOwner functionality

src/Morpho.sol:88–94

DESCRIPTION

`setOwner` only checks that `newOwner != owner` but does not reject `address(0)`. If the current owner mistakenly (or maliciously, e.g. in a renounce attempt) sets the owner to the zero address, every `onlyOwner` function (`enableIrm`, `enableLtv`, `setFee`, `setFeeRecipient`, `setOwner` itself) becomes permanently inaccessible since no future caller can satisfy `msg.sender == owner`.

IMPACT

Permanent loss of governance over IRM/LLTV whitelisting, fee configuration, and fee recipient updates. While it does not directly endanger already-supplied funds, it freezes protocol-level parameter management forever.

RECOMMENDATION

Add an explicit zero-address check, e.g.:

```
```solidity
function setOwner(address newOwner) external onlyOwner {
 require(newOwner != address(0), ErrorsLib.ZERO_ADDRESS);
 require(newOwner != owner, ErrorsLib.ALREADY_SET);
 owner = newOwner;
 emit EventsLib.SetOwner(newOwner);
}
```
```

```
function setOwner(address newOwner) external onlyOwner {
    require(newOwner != owner, ErrorsLib.ALREADY_SET);
    owner = newOwner;
    emit EventsLib.SetOwner(newOwner);
}
```

F-002

Low

setFeeRecipient allows zero address and lets a newly-set recipient claim unaccrued pending fees

src/Morpho.sol:131–137

DESCRIPTION

``setFeeRecipient`` performs no zero-address check and does not force interest accrual across all markets before switching recipients. As documented in the interface comments, changing the fee recipient allows the **new** recipient to claim fee shares that accrued (but were not yet materialized via ``_accrueInterest``) under the **previous** recipient's tenure, since ``_accrueInterest`` mints fee shares to whichever address is currently ``feeRecipient`` at call time.

IMPACT

If the owner is compromised or acts maliciously, fees intended for the legitimate recipient across all active markets can be redirected to a new address, or lost entirely if set to ``address(0)``.

RECOMMENDATION

Add ``require(newFeeRecipient != address(0), ErrorsLib.ZERO_ADDRESS)``. Consider a governance pattern that accrues interest on all live markets (or documents an operational runbook to do so) prior to recipient rotation; at minimum, keep the current explicit warning in NatSpec and ensure it is acted upon operationally.

```
function setFeeRecipient(address newFeeRecipient) external onlyOwner {
    require(newFeeRecipient != feeRecipient, ErrorsLib.ALREADY_SET);
    feeRecipient = newFeeRecipient;
    emit EventsLib.SetFeeRecipient(newFeeRecipient);
}
```

F-003

Low

Centralization risk: owner controls IRM/LLTV whitelisting, market fees, and fee recipient with no timelock

src/Morpho.sol:89–138

DESCRIPTION

A single ``owner`` EOA/contract can enable arbitrary Interest Rate Model contracts (``enableIrm``), enable arbitrary LLTVs up to ``WAD`` (``enableLltv``), set per-market fees up to ``MAX_FEE`` (25%), and redirect ``feeRecipient``. There is no on-chain timelock, multisig enforcement, or governance delay visible in this contract; all protections depend on off-chain key management for the ``owner`` address.

IMPACT

A compromised or malicious owner key can whitelist a malicious IRM (which could manipulate ``borrowRate`` to cause runaway interest/overflow), extract maximum fees from all markets, or redirect fee flows. It cannot directly steal already-deposited principal, but can materially harm market economics and depositor returns.

RECOMMENDATION

Operate the ``owner`` address as a timelocked multisig/DAO-governed contract rather than an EOA, and ensure IRM/LLTV additions go through public review before being enabled (standard practice already followed by Morpho governance off-chain).

```
function enableIrm(address irm) external onlyOwner {
    require(!isIrmEnabled[irm], ErrorsLib.ALREADY_SET);
    isIrmEnabled[irm] = true;
    emit EventsLib.EnableIrm(irm);
}
```

F-004

Low

No staleness, bounds, or revert-handling check on oracle price feed

src/Morpho.sol:349–379

DESCRIPTION

``_isHealthy`` and ``liquidate`` call ``IOracle(marketParams.oracle).price()`` directly with no check on staleness, zero/garbage values, or extreme magnitudes. The protocol explicitly documents this as the market creator's responsibility ('the oracle should return a price with the correct scaling... should not revert... should not return a very high price'), but the on-chain code itself has no defensive guard.

IMPACT

If a market is created pointing to a faulty, stale, or manipulable oracle (e.g., a thinly-liquidity Uniswap spot price without TWAP, or a Chainlink feed without staleness checks), borrowers can be wrongfully liquidated or can borrow against an inflated collateral valuation, leading to protocol insolvency for that specific market. This does not affect markets using sound oracles but is a systemic risk vector per-market.

RECOMMENDATION

This is an accepted architectural trade-off for a permissionless, isolated-market design; no code change is required in Morpho core. Ensure front-ends and market-creation tooling strongly warn integrators to use oracles with staleness checks, circuit breakers, and manipulation-resistant price sources (e.g., Chainlink with heartbeat checks, or TWAP oracles), and audit each oracle adapter independently before whitelisting its LLTV/IRM combination.

```
uint256 collateralPrice = IOracle(marketParams.oracle).price();
require(!_isHealthy(marketParams, id, borrower, collateralPrice), ErrorsLib.HEALTHY_POSITION);
```

F-005 · SWC-121

Low

EIP-712 domain separator fixed at deployment with no re-derivation on chain-fork

src/Morpho.sol:69–73

DESCRIPTION

``DOMAIN_SEPARATOR`` is computed once in the constructor using ``block.chainid`` at deploy time and stored as ``immutable``. If Ethereum mainnet undergoes a contentious hard fork that results in two chains sharing the same chain ID (or if this bytecode is naively redeployed on a new chain with the same chain ID by mistake), ``setAuthorizationWithSig`` signatures become replayable across those chains. This is explicitly acknowledged in the NatSpec warning.

IMPACT

In a chain-split scenario, an ``Authorization`` signature valid on one fork could be replayed on the other fork, potentially authorizing an address to manage positions without the authorizer's consent on the unintended chain.

RECOMMENDATION

This is a known, low-likelihood, documented trade-off common to immutable EIP-712 domain separators. If re-deployment is ever considered, use a dynamic ``DOMAIN_SEPARATOR`` that recomputes based on current ``block.chainid`` when it differs from the cached value (OpenZeppelin's ``EIP712`` pattern), trading a small gas cost for fork safety.

```
DOMAIN_SEPARATOR = keccak256(abi.encode(DOMAIN_TYPEHASH, block.chainid, address(this)));
```

Floating pragma in library and interface files

src/libraries/MathLib.sol:3–3

DESCRIPTION

Several in-scope library and interface files use floating pragmas (`^0.8.0`` for libraries, ``>=0.5.0`` for interfaces) rather than being pinned to the exact compiler version used for the main ``Morpho.sol`` contract (``0.8.19``). While the deployed bytecode is compiled with a fixed version, floating pragmas in the source can lead to inconsistent compiler behavior if these files are reused/compiled independently in other projects.

IMPACT

Low, purely a source-hygiene/reproducibility concern; does not affect the already-deployed bytecode's behavior.

RECOMMENDATION

Pin all first-party library files to the exact compiler version used for deployment, e.g. ``pragma solidity 0.8.19;``, consistent with ``Morpho.sol``.

```
pragma solidity ^0.8.0;
```

Gas Optimizations

Cache repeated `market[id]` storage reads/writes into a local storage pointer

src/Morpho.sol:205–240

Functions like `supply`, `withdraw`, `borrow`, `repay`, and `liquidate` read and write `market[id]` multiple times via the mapping. Using `Market storage m = market[id];` once per function and operating on `m` reduces redundant keccak256/mapping lookups and can save gas on repeated field access.

Avoid recomputing `marketParams.id()` when already available

src/Morpho.sol:160–420

Several external functions already compute `Id id = marketParams.id();` once and reuse it correctly; ensure this pattern is consistently used everywhere `MarketParamsLib.id` is invoked to avoid duplicate keccak256 computations within the same call (already mostly followed, no action needed beyond maintaining discipline in future changes).

Appendix - Metadata

| | |
|----------|--|
| Contract | Morpho |
| Address | 0xBBBBBbbBBb9cC5e90e3b3Af64bdAF62C37EEFFCb |
| Chain | Ethereum |
| Compiler | v0.8.19+commit.7dd6d404 |
| Audit ID | portfolio-morpho-blue |
| Date | 10/7/2026 |

Files analyzed

```
[user] src/Morpho.sol
[user] src/interfaces/IMorpho.sol
[user] src/interfaces/IMorphoCallbacks.sol
[user] src/interfaces/IIm.sol
[user] src/interfaces/IERC20.sol
[user] src/interfaces/IOracle.sol
[user] src/libraries/ConstantsLib.sol
[user] src/libraries/UtilsLib.sol
[user] src/libraries/EventsLib.sol
[user] src/libraries/ErrorsLib.sol
[user] src/libraries/MathLib.sol
[user] src/libraries/SharesMathLib.sol
[user] src/libraries/MarketParamsLib.sol
[user] src/libraries/SafeTransferLib.sol
```

MANUAL AUDIT UPGRADE

Need a deeper audit?

\$5,000

Pay in ETH, SOL, or USDT. Detailed manual review by Crypto Hawking, ETHDenver 2025 Winner. Comprehensive findings, remediation steps, and post-fix re-review. Report delivered in 3 business days.

[Request Manual Audit '](https://cryptohawking.com/audit/manual?ref=portfolio-morpho-blue)

<https://cryptohawking.com/audit/manual?ref=portfolio-morpho-blue>

Need it built, not just audited?

I also build production smart contracts end to end, token launches, DeFi protocols, staking, and full DEX stacks (v2, v3 concentrated liquidity, and beyond), plus fractional CTO engagements for Web3 teams. Fixes for the findings in this report can be scoped in the same engagement.

<https://cryptohawking.com/services>

WhatsApp (preferred): +92 322 4274236, <https://wa.me/923224274236>